

Задание 1.2

Установка и настройка программного обеспечения компьютерной аудитории

Цель работы

Подготовить скрипт PowerShell для автоматизации установки и настройки программного обеспечения учебного компьютера, проверить его под новой учётной записью и описать порядок работы.

Участники и распределение работы

Качков К.Е. — сделал скрипт.

Парванян Г.С. — протестировал скрипт, зайдя с нового пользователя.

Момот М.-М. М. — подготовила отчёт.

Порядок выполнения

Этап 1 выполняется в PowerShell от имени администратора: установка WSL и инструментов сборки C++, затем перезагрузка компьютера.

Этап 2 выполняется в обычном PowerShell под рабочей учётной записью: установка приложений, настройка сред программирования и расширений Visual Studio Code.

Этап 3 выполняется от имени администратора: создание резервного образа на отдельном физическом диске. Параметр BackupDrive задаёт диск для резервной копии.

Примеры запуска опубликованного файла:

```
powershell -ExecutionPolicy Bypass -File .\Скрипт.ps1 -Stage 1
```

```
powershell -ExecutionPolicy Bypass -File .\Скрипт.ps1 -Stage 2
```

```
powershell -ExecutionPolicy Bypass -File .\Скрипт.ps1 -Stage 3 -BackupDrive "E:"
```

Устройство скрипта

Скрипт разделяет действия с административными правами и настройку профиля пользователя. Независимые операции обёрнуты в обработку ошибок: сбой одной установки записывается в журнал и не прекращает остальные задачи. Журналы сохраняются в папке LabSetup внутри LOCALAPPDATA.

Приложения устанавливаются через WinGet; дополнительно настраиваются WSL 2, Ubuntu, MSYS2 UCRT64, Rust, Julia, Python, Git и расширения VS Code. Установщики Телемоста и Jazz загружаются по прямым ссылкам. Для Jazz предусмотрено завершение мастера установки пользователем.

Перед резервным копированием проверяется, что целевой диск существует и находится на другом физическом накопителе. Команда wadmin создаёт образ системного и критических томов. Коды завершения: 0 — команды выполнены без зарегистрированных ошибок, 1 — обнаружены ошибки, 3010 — требуется перезагрузка.

Проверка и результат

Парванян Г.С. выполнил тестирование скрипта, войдя в систему под новым пользователем. Подготовлены скрипт автоматизации и отчёт с описанием этапов и полным листингом кода.

Листинг скрипта

```
#requires -Version 5.1
# Windows x64, один компьютер и одна рабочая учетная запись.
# Stage 1: PowerShell от администратора – WSL и инструменты C++, затем перезагрузка.
# Stage 2: обычный PowerShell – программы и настройки текущего пользователя.
# Stage 3: PowerShell от администратора – резервный образ на отдельный диск.
# Запуск из папки с файлом: powershell -ExecutionPolicy Bypass -File .\Install-Lab.ps1 -Stage 1
# Для следующих этапов замените номер; для образа добавьте -BackupDrive "E:".
param(
    [Parameter(Mandatory = $true)]
    [ValidateSet(1, 2, 3)]
    [int]$Stage,
    [ValidatePattern('^[A-Za-z]:$')]
    [string]$BackupDrive = 'E:'
)

# Ошибки PowerShell обрабатываем через try/catch, коды внешних программ – отдельно.
$ErrorActionPreference = 'Stop'
if (Test-Path variable:PSNativeCommandUseErrorActionPreference) { $PSNativeCommandUseErrorActionPreference = $false }
# TLS 1.2 нужен для HTTPS-загрузок в Windows PowerShell 5.1.
[Net.ServicePointManager]::SecurityProtocol = [Net.ServicePointManager]::SecurityProtocol -bor
[Net.SecurityProtocolType]::Tls12
# Общие переменные: функции записывают сюда ошибки и необходимость перезагрузки.
$script:Failures = @()
$script:RestartNeeded = $false
# Установщики и журнал сохраняем в доступную текущему пользователю папку.
$folder = Join-Path $env:LOCALAPPDATA 'LabSetup'
$log = Join-Path $folder ('stage-{0}-{1}.log' -f $Stage, (Get-Date -Format 'yyyyMMdd-HH:mm:ss'))
New-Item -ItemType Directory -Path $folder -Force | Out-Null

# Каждая независимая задача записывает ошибку, но не мешает установке остальных программ.
function Run-Task([string]$Name, [scriptblock]$Action) {
    Write-Host "`n>>> $Name" -ForegroundColor Cyan
    try { & $Action }
    catch {
        $script:Failures += "${Name}: ${_.Exception.Message}"
        Write-Warning $script:Failures[-1]
    }
}

# Ненулевой код внешней команды превращаем в ошибку для Run-Task или общего catch.
function Run-Command([string]$File, [string[]]$Arguments = @()) {
    & $File @Arguments | Out-Host
    if ($LASTEXITCODE -ne 0) { throw "$File – код $LASTEXITCODE." }
}

# Служебные команды WSL выводят UTF-16: читаем обе выходные очереди в этой кодировке.
# Иначе Windows PowerShell 5.1 может записать в журнал нечитаемый текст.
function Run-Wsl([string[]]$Arguments, [switch]$Capture) {
    $start = New-Object System.Diagnostics.ProcessStartInfo
    $start.FileName = 'wsl.exe'
    $start.Arguments = $Arguments -join ' '
    $start.UseShellExecute = $false
    $start.CreateNoWindow = $true
    $start.RedirectStandardOutput = $true
    $start.RedirectStandardError = $true
    $start.StandardOutputEncoding = [Text.Encoding]::Unicode
    $start.StandardErrorEncoding = [Text.Encoding]::Unicode
    $start.EnvironmentVariables.Remove('WSL_UTF8') # Кодировка по умолчанию только для дочернего процесса.
    $process = New-Object System.Diagnostics.Process
    $process.StartInfo = $start
    try {
        [void]$process.Start()
        $stdout = $process.StandardOutput.ReadToEndAsync()
    }
```

```

        $stderr = $process.StandardError.ReadToEndAsync()
        $process.WaitForExit()
        $output = $stdout.Result
        $errorText = $stderr.Result
        if ($output) { Write-Host $output.TrimEnd() }
        if ($errorText) { Write-Host $errorText.TrimEnd() }
        if ($process.ExitCode -notin @(0, 3010)) {
            throw "wsl $($Arguments -join ' ') - код $($process.ExitCode). $errorText $output"
        }
        if ($process.ExitCode -eq 3010) {
            $script:RestartNeeded = $true
            if ($Stage -eq 2) { throw 'WSL требует перезагрузки. После нее повторите этап 2.' }
        }
        if ($Capture) { return @($output -split '\r?\n' | ForEach-Object { $_.Trim() } | Where-Object { $_ })
    }
    finally { $process.Dispose() }
}

# Обновляем PATH текущего окна, чтобы видеть команды установленных программ.
function Refresh-Path {
    $env:Path = [Environment]::GetEnvironmentVariable('Path', 'Machine') + ';' +
        [Environment]::GetEnvironmentVariable('Path', 'User')
}

# Добавляем каталог в пользовательский PATH без повторяющихся записей.
function Add-UserPath([string]$Path) {
    if (-not (Test-Path -LiteralPath $Path -PathType Container)) { throw "Не найден каталог $Path." }
    $old = [string][Environment]::GetEnvironmentVariable('Path', 'User')
    if (@($old -split ';' | ForEach-Object { $_.TrimEnd('\') }) -notcontains $Path.TrimEnd('\')) {
        [Environment]::SetEnvironmentVariable('Path', ($old.TrimEnd(';') + ';' + $Path).TrimStart(';'),
        'User')
    }
    Refresh-Path
}

# Отличаем отсутствие программы от сбоя WinGet или его источника.
function Test-Program([string]$Id) {
    & winget.exe list --id $Id --exact --source winget --accept-source-agreements --disable-interactivity |
    Out-Host
    if ($LASTEXITCODE -eq 0) { return $true }
    if ($LASTEXITCODE -eq -1978335212) { return $false } # WinGet: пакет не установлен (0x8A150014).
    throw "Не удалось проверить ${Id}: winget вернул $LASTEXITCODE."
}

# Пропускаем установленное ПО; после новой установки повторно проверяем наличие.
function Install-Program([string]$Id) {
    if (Test-Program $Id) { Write-Host "Уже установлено: $Id"; return }
    # --exact выбирает точный ID; --silent скрывает мастер; соглашения принимаются автоматически.
    & winget.exe install --id $Id --exact --source winget --silent --accept-package-agreements --accept-
    source-agreements --disable-interactivity | Out-Host
    $code = $LASTEXITCODE
    if ($code -in @(3010, -1978334967, -1978334966)) { # Перезагрузка требуется во время/после установки.
        $script:RestartNeeded = $true
        throw 'Перезагрузите компьютер и повторите этап 2 для завершения установки и проверки.'
    }
    if ($code -ne 0) { throw "Установка ${Id}: код $code." }
    if (-not (Test-Program $Id)) { throw "Установка $Id не подтверждена." }
}

# Файлы загружаются с официальных сайтов. Проверяется формат, чтобы не запустить HTML-страницу.
function Download-Installer([string]$Url, [string]$Name) {
    $file = Join-Path $folder $Name
    $ProgressPreference = 'SilentlyContinue' # Ускоряет загрузку больших файлов в PowerShell 5.1.
    Invoke-WebRequest -UseBasicParsing -Uri $Url -OutFile $file
    $stream = [IO.File]::OpenRead($file)
    try { $bytes = New-Object byte[] 8; $count = $stream.Read($bytes, 0, 8) }
    finally { $stream.Dispose() }
    # Начальные байты формата MSI/EXE; это не проверка цифровой подписи.
    $expected = if ($Name -like '*.msi') { 'D0-CF-11-E0-A1-B1-1A-E1' } else { '4D-5A' }
    if ($count -lt 8 -or -not ([BitConverter]::ToString($bytes).StartsWith($expected))) {
        throw "Вместо $Name получен файл неподходящего формата."
    }
    return $file
}

```

```

# Ставим WSL напрямую из официального MSI, без wsl --install и Microsoft Store.
function Install-Wsl {
    if ([Environment]::OSVersion.Version.Build -lt 19041) { throw 'Для WSL нужны Windows 10 сборки 19041 или
    новее либо Windows 11.' }
    Write-Host 'WSL: установка из MSI Microsoft и включение Virtual Machine Platform.'
    $release = Invoke-RestMethod -Uri 'https://api.github.com/repos/microsoft/WSL/releases/latest'
    $asset = $release.assets | Where-Object { $_.name -match '^wsl\[0-9.\]+\x64\.msi$' } | Select-Object -
    First 1
    if (-not $asset) { throw 'В выпуске Microsoft WSL не найден установщик x64.' }
    Write-Host "Загрузка WSL $($release.tag_name)..."
    $setup = Download-Installer $asset.browser_download_url 'wsl-x64.msi'
    $signature = Get-AuthenticodeSignature -LiteralPath $setup
    if ($signature.Status -ne 'Valid' -or $signature.SignerCertificate.Subject -notmatch 'O=Microsoft
    Corporation') { throw 'Не подтверждена подпись Microsoft у WSL.' }

    # Устанавливаем WSL вместе с ядром Linux; Windows Installer пишет отдельный журнал.
    $msilog = Join-Path $folder 'wsl-install.log'
    $arguments = '/i "{0}" /qn /norestart /L*v "{1}"' -f $setup, $msilog
    $process = Start-Process -FilePath 'msiexec.exe' -ArgumentList $arguments -WindowStyle Hidden -Wait -
    PassThru
    if ($process.ExitCode -notin @(0, 3010)) { throw "Установка WSL из MSI: код $($process.ExitCode). Журнал:
    $msilog" }
    if ($process.ExitCode -eq 3010) { $script:RestartNeeded = $true }

    # Компонент виртуализации Windows необходим для WSL 2; применяется после перезагрузки.
    & dism.exe /Online /Enable-Feature /FeatureName:VirtualMachinePlatform /All /NoRestart /English | Out-
    Host
    if ($LASTEXITCODE -notin @(0, 3010)) { throw "Включение Virtual Machine Platform: код $LASTEXITCODE.
    Журнал: $env:windir\Logs\DISM\dism.log" }
    $script:RestartNeeded = $true
}

# Телемост и Jazz ищем по названию в реестре установленных программ Windows.
function Test-InstalledApp([string]$Pattern) {
    $keys = @( 'HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*',
    'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*',
    'HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*' )
    return (@(Get-ItemProperty -Path $keys -ErrorAction SilentlyContinue |
    Where-Object { $_.DisplayName -match $Pattern }).Count -gt 0)
}

# Rust под Windows нужны инструменты C++ и Windows SDK; ставим их через Build Tools.
function Install-BuildTools {
    $vsFolder = Join-Path $(env:ProgramFiles(x86)) 'Microsoft Visual Studio\Installer'
    $vswhere = Join-Path $vsFolder 'vswhere.exe'
    $installed = $null
    if (Test-Path -LiteralPath $vswhere) {
        $installed = & $vswhere -latest -products Microsoft.VisualStudio.Product.BuildTools -version
        '[17.0,18.0)' -property installationPath
        if ($LASTEXITCODE -ne 0) { throw 'Не удалось проверить Build Tools.' }
    }
    # Имеющуюся установку дополняем компонентами; иначе скачиваем установщик Microsoft.
    if ($installed) {
        $setup = Join-Path $vsFolder 'setup.exe'
        $arguments = 'modify --installPath "{0}" --channelId VisualStudio.17.Release --quiet --norestart --
        add Microsoft.VisualStudio.Workload.VCTools --includeRecommended' -f ([string]$installed).Trim()
    }
    else {
        $setup = Download-Installer 'https://aka.ms/vs/17/release/vs_buildtools.exe' 'vs-buildtools.exe'
        $signature = Get-AuthenticodeSignature -LiteralPath $setup
        if ($signature.Status -ne 'Valid' -or $signature.SignerCertificate.Subject -notmatch 'O=Microsoft
        Corporation') { throw 'Не подтверждена подпись Microsoft у Build Tools.' }
        $arguments = '--quiet --wait --norestart --add Microsoft.VisualStudio.Workload.VCTools --
        includeRecommended'
    }
    $process = Start-Process -FilePath $setup -ArgumentList $arguments -WorkingDirectory $folder -WindowStyle
    Hidden -Wait -PassThru
    if ($process.ExitCode -notin @(0, 3010)) { throw "Build Tools: код $($process.ExitCode)." }
    if ($process.ExitCode -eq 3010) { $script:RestartNeeded = $true }
}

# Записываем вывод в журнал и проверяем права перед выполнением выбранного этапа.
$exitCode = 0

```

```

Start-Transcript -Path $log | Out-Null
try {
    if (-not [Environment]::Is64BitProcess -or $env:PROCESSOR_ARCHITECTURE -ne 'AMD64') { throw 'Нужны
Windows x64 и 64-разрядный PowerShell.' }
    $principal = New-Object
Security.Principal.WindowsPrincipal([Security.Principal.WindowsIdentity]::GetCurrent())
    $admin = $principal.IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
    if ($Stage -in @(1, 3) -and -not $admin) { throw 'Для этапов 1 и 3 откройте PowerShell от имени
администратора.' }
    # Пользовательские программы, Ubuntu и расширения должны попасть в рабочий профиль.
    if ($Stage -eq 2 -and $admin) { throw 'Для этапа 2 откройте обычный PowerShell под своей учетной
записью.' }

    # Этап 1: системная подготовка. Ubuntu установим после перезагрузки на этапе 2.
    if ($Stage -eq 1) {
        Install-Wsl
        Run-Task 'Build Tools для Rust' { Install-BuildTools }
        Write-Host 'Перезагрузите компьютер. Затем запустите этап 2 в обычном PowerShell.'
    }

    # Этап 2: установка и настройка ПО для текущего пользователя.
    if ($Stage -eq 2) {
        Refresh-Path
        if (-not (Get-Command winget.exe -ErrorAction SilentlyContinue)) { throw 'Установите/обновите
«Установщик приложений» Microsoft из Store и откройте PowerShell заново.' }
        Run-Task 'WSL 2' {
            Run-Wsl @('--version') # WSL уже установлен из MSI на этапе 1.
            Run-Wsl @('--set-default-version', '2')
            $installed = @(Run-Wsl @('--list', '--quiet') -Capture)
            foreach ($distro in @('Ubuntu-22.04', 'Ubuntu-24.04')) {
                if ($installed -contains $distro) { Run-Wsl @('--set-version', $distro, '2') }
                else { Run-Wsl @('--install', '-d', $distro, '--no-launch', '--web-download') }
            }
        }
        # 27 программ из задания. WSL, Телемост и Jazz обрабатываются отдельно.
        $programs = @(
            'Microsoft.VisualStudioCode', 'Docker.DockerDesktop', 'JetBrains.PyCharm.Community',
            'Git.Git', 'GitHub.GitHubDesktop', 'MaximaTeam.Maxima', 'KNIMEAG.KNIMEAnalyticsPlatform',
            'GIMP.GIMP.3', 'Julialang.Julia', 'Python.Python.3.13', 'Rustlang.Rustup', 'MSYS2.MSYS2',
            'Zettlr.Zettlr', 'MiKTeX.MiKTeX', 'Chocolatey.Chocolatey', 'TeXstudio.TeXstudio',
            'Anaconda.Anaconda3', 'FarManager.FarManager', 'SumatraPDF.SumatraPDF', 'Google.Chrome',
            'Flameshot.Flameshot', 'Qalculate.Qalculate', 'TheBrowserCompany.Arc', '7zip.7zip',
            'Mozilla.Firefox', 'Yandex.Browser', 'Microsoft.Edge'
        )
        foreach ($program in $programs) { Run-Task $program { Install-Program $program } }
        Refresh-Path

        # Устанавливаем GCC/G++ и GDB, затем делаем их доступными из терминала и VS Code.
        Run-Task 'MSYS2 UCRT64: компилятор и отладчик' {
            $bash = 'C:\msys64\usr\bin\bash.exe'
            if (-not (Test-Path -LiteralPath $bash)) { throw 'Не найден C:\msys64. Проверьте установку
MSYS2.' }
            # Первый проход может закрыть оболочку при обновлении ядра MSYS2; второй обязателен.
            & $bash -lc 'pacman --noconfirm -Syuu' | Out-Host
            Write-Host "Первый проход MSYS2: код $LASTEXITCODE. Выполняется второй проход."
            Run-Command $bash @('-lc', 'pacman --noconfirm -Syuu')
            Run-Command $bash @('-lc', 'pacman -S --needed --noconfirm base-devel mingw-w64-ucrt-x86_64-
toolchain')
            Add-UserPath 'C:\msys64\ucrt64\bin'
            Run-Command 'g++.exe' @('--version')
            Run-Command 'gdb.exe' @('--version')
        }
        # Сохраняем выбранный набор инструментов Rust; stable нужен, только если активного нет.
        Run-Task 'Rust' {
            Add-UserPath (Join-Path $env:USERPROFILE '.cargo\bin')
            & rustup.exe show active-toolchain | Out-Host
            if ($LASTEXITCODE -ne 0) { Run-Command 'rustup.exe' @('default', 'stable') }
            Run-Command 'rustc.exe' @('--version')
            Run-Command 'cargo.exe' @('--version')
        }
        # Установщик Julia не всегда добавляет команду в PATH — при необходимости делаем это сами.
        Run-Task 'Julia' {
            if (-not (Get-Command julia.exe -ErrorAction SilentlyContinue)) {
                $julia = Get-ChildItem -Path "$env:LOCALAPPDATA\Programs\Julia*\bin\julia.exe" -ErrorAction
SilentlyContinue | Sort-Object LastWriteTime -Descending | Select-Object -First 1
                if (-not $julia) { throw 'Не найден julia.exe. Проверьте установку Julia.' }
            }
        }
    }
}

```

```

        Add-UserPath $julia.DirectoryName
    }
    Run-Command 'julia.exe' @('--version')
}
Run-Task 'Python' { Run-Command 'py.exe' @('-3.13', '--version') }
Run-Task 'Git' { Run-Command 'git.exe' @('--version') }
# Расширения для Python, C++, контейнеров, веб-разработки, Git, Julia, Rust и WSL.
$extensions = @(
    'ms-python.python', 'ms-python.vscode-pylance', 'ms-vscode.cpptools',
    'ms-azuretools.vscode-containers', 'ritwickdey.LiveServer', 'esbenp.prettier-vscode',
    'eamodio.gitlens', 'julialang.language-julia', 'rust-lang.rust-analyzer',
    'ms-vscode-remote.remote-wsl'
)
foreach ($extension in $extensions) {
    Run-Task "VS Code: $extension" { Run-Command 'code.cmd' @('--install-extension', $extension) }
}
# Официальный MSI: https://yandex.ru/support/yandex-360/business/admin/ru/telemost/telemost-msi
Run-Task 'Яндекс Телемост' {
    if (-not (Test-InstalledApp 'Telemost|Телемост')) {
        $msi = Download-Installer 'https://webdav.yandex.ru/share/dist/YTelemostSetup.msi'
'telemost.msi'
        # Тихая установка в текущий профиль, без перезагрузки и запуска приложения.
        $arguments = '/i "{0}" /qn /norestart ALLUSERS=2 MSIINSTALLPERUSER="1" SKIP_LAUNCH=1' -f $msi
        $process = Start-Process -FilePath 'msiexec.exe' -ArgumentList $arguments -WindowStyle Hidden
-Wait -PassThru
        if ($process.ExitCode -notin @(0, 3010)) { throw "Телемост: код $($process.ExitCode)." }
        if ($process.ExitCode -eq 3010) { $script:RestartNeeded = $true }
        if (-not (Test-InstalledApp 'Telemost|Телемост')) { throw 'Установка Телемоста не
подтверждена.' }
    }
}
# Официальная ссылка: https://developers.sber.ru/help/jazz
Run-Task 'Jazz' {
    $pattern = 'Sber.?Jazz|Salute.?Jazz|Сбер.?Джаз|Салют.?Джаз|^Jazz$|^Джаз$'
    if (-not (Test-InstalledApp $pattern)) {
        $exe = Download-Installer 'https://dl.salutejazz.ru/desktop/latest/jazz.exe' 'jazz.exe'
        Write-Host 'Завершите мастер установки Jazz. Если приложение откроется, закройте его для
продолжения.'
        # Для Jazz оставляем обычный мастер: проверенные ключи тихой установки не заданы.
        $process = Start-Process -FilePath $exe -Wait -PassThru
        if ($process.ExitCode -notin @(0, 3010)) { throw "Jazz: код $($process.ExitCode)." }
        if ($process.ExitCode -eq 3010) { $script:RestartNeeded = $true }
        if (-not (Test-InstalledApp $pattern)) { throw 'Установка Jazz не подтверждена.' }
    }
}
Write-Host 'После установки откройте обе Ubuntu, создайте Linux-пользователей и введите exit.'
Write-Host 'Откройте Docker Desktop и завершите первый запуск. Затем проверьте: wsl -l -v и docker
info.'
Write-Host 'Для C++ в VS Code выберите компилятор C:\msys64\ucrt64\bin\g++.exe.'
}

# Этап 3: создаём образ на другом физическом диске; диск не форматируем.
if ($Stage -eq 3) {
    if ($BackupDrive -eq $env:SystemDrive -or -not (Test-Path -LiteralPath "$BackupDrive\")) { throw
'Укажите существующий отдельный диск для резервной копии.' }
    $target = Get-Partition -DriveLetter $BackupDrive[0]
    $system = Get-Partition -DriveLetter $env:SystemDrive[0]
    if ($target.DiskNumber -eq $system.DiskNumber -or $target.IsBoot -or $target.IsSystem) { throw
'Резервную копию нужно сохранять на отдельном физическом диске.' }
    Write-Host 'В образ войдут системный и критические тома. Программы и данные на других томах не
включаются.'
    # -allCritical включает критические тома; -vssCopy не меняет историю других VSS-копий.
    Run-Command 'wbadmin.exe' @('start', 'backup', "-backupTarget:$BackupDrive", "-
include:$env:SystemDrive", '-allCritical', '-vssCopy', '-quiet')
    # Показываем каталог копий; это не заменяет проверку восстановления.
    Run-Command 'wbadmin.exe' @('get', 'versions', "-backupTarget:$BackupDrive")
}
}
catch { $script:Failures += $_.Exception.Message; Write-Warning $_.Exception.Message }
# Завершаем журнал даже при ошибке. Коды: 0 – без ошибок, 1 – ошибки, 3010 – перезагрузка.
finally {
    if ($script:Failures.Count) {
        Write-Host "`nНе выполнено:" -ForegroundColor Yellow
        $script:Failures | ForEach-Object { Write-Host "- $_" }
        Write-Host 'Устраните причины и повторите этот этап. Уже установленные программы будут пропущены.'
        $exitCode = 1
    }
}

```

```
else { Write-Host "`nКоманды этапа выполнены. Учтите сообщения о первом запуске приложений." }  
if ($script:RestartNeeded) {  
    Write-Host 'Нужна перезагрузка компьютера.' -ForegroundColor Yellow  
    if (-not $script:Failures.Count) { $exitCode = 3010 }  
}  
Write-Host "Журнал: $log"  
Stop-Transcript | Out-Null  
}  
exit $exitCode
```